Buffer Management Simulator

- **Background**

In the session on buffer management, we have the following exercise:

Consider the following buffer pool with four frames.

pin	*dirty*	*page*
3	*1*	*A*
1	*0*	*B*
1	*1*	*C*

Find the status of the buffer pool after the following steps:

1. *The page B is requested.*
2. *The page A is released.*
3. *The page C is released, where it is not modified.*
4. *The page X is requested.*
5. *The page H is requested.*

Assume that each disk access takes 1 I/O cost. What is the total I/O cost after these five steps?

In this coding project, you need to implement a program that can solve such exercises.

- **Description**

Your program should read a sequence of page requests similar to the exercise described and provide corresponding instructions for each request. Unlike the exercise, which requires you to draw the status of the buffer pool after each request, your program should output page I/O instructions as follows:

1. Page Read: Load a page from disk into the buffer pool.
2. Page Write: Write the contents of a page in the buffer pool back to disk.
3. Remove a Page: Remove a page from the buffer pool.

Note that the third instruction does not incur any I/O cost. However, if the page is dirty, you should write it back to disk before removing it from the buffer pool.

For a page X, its lifecycle in the buffer pool is as follows:

Stage 1: Loading into Buffer Pool

- Initially, page X resides on disk.
- When there is a read request for page X (e.g., a program wants to read its contents), it needs to be loaded into the buffer pool (main memory).

Stage 2: Usage in Buffer Pool

- While page X is in the buffer pool, multiple programs may use it, which is tracked by the pin count. The pin count of page X indicates the number of programs using it.
- The pin count increases by 1 when page X is requested by a program and decreases by 1 when a program releases it. Note that "release" here means a program stops using page X, not removing it from the buffer pool.

Stage 3: Buffer Replacement

- The buffer pool size is limited, and in this project, we assume it can contain at most N pages.
- When the buffer pool is full and a new page needs to be loaded, buffer replacement occurs.
- Pages in the buffer pool can be categorized as follows:
    - Pages with a pin count > 0: These pages are in use by programs and cannot be removed.
    - Pages with a pin count = 0: These pages are not currently in use and can be removed. If a page is dirty, it must be written back to the disk before removal.
- When buffer replacement is needed:
    - If there are no pages with a pin count of 0, the new page read request fails.
    - If there are k pages with a pin count of 0, choose one for replacement according to the buffer replacement policy.

- **Your Task**

Implement a program in C/C++, Java, or Python to simulate the buffer management process using the Least Recently Used (LRU) policy. Your program should read a sequence of page release and request operations and output the corresponding page I/O operations for each page read request.

- **Detailed Setting:**

Three parameters: N, M, and Q:

1. N: The size of the buffer pool.
2. M: The total number of pages, each denoted by an integer page ID (pid). The page IDs range from 1 to M (i.e., 0 < pid <= M).
3. Q: The number of page release and request operations.

In this coding project, we assume that all page read requests can be satisfied. In other words, when buffer replacement occurs, the input data guarantees that there is at least one page with a pin count of 0.

The LRU policy is adopted in this project. This means that when buffer replacement happens, you should throw out the page that has not been **requested** for the longest time. In the setting of this course, the page release(*) operation does not update the time stamp.

- **Input Format**

Read data from the standard input (keyboard).

The first line contains three integers N, M and Q.

Then Q lines follow, each line contains one page release / request operation in one of the following three formats:

Code	Info
`request pid`	The page pid is requested.
`release pid`	The page pid is released **with** modification.
`release* pid`	The page pid is released **without** modification

- **Output Format**

Write results to the standard output (screen).

For each page request "request pid", output corresponding page I/O instructions if any.

Each line contains one instruction in the following format:

Code	Info
`read pid`	Load page pid from disk to the buffer pool.
`write pid`	Write the contents of page pid (in the buffer pool) back to disk.
`remove pid`	Remove page pid from the buffer pool.

- **Sample 1 Input**

```
4 10 16
request 1
release 1
request 1
request 1
request 1
request 2
request 2
release* 2
request 3
request 3
release 3
request 2
```

```
release 1
release* 3
request 8
request 10
```

- **Sample 1 Output**

```
read 1
read 2
read 3
read 8
write 3
remove 3
read 10
```

- **Sample 1 Explanation**

This sample shows the exercise given in the background part.

First 11 operations construct the initial state shown in the exercise.

The last 5 operations correspond to the given steps.

Page A, B, C, X and H maps to pid 1, 2, 3, 8 and 10 respectively.

When page H (pid=10) is requested (the last operation), buffer replacement happens.

- Page C (3) is the only one whose pin count is 0, so remove page C for replacement.
- Since page C is dirty, we should write page C back to disk before remove it.
- After remove page C, load page H into the buffer pool.

Therefore, we have a sequence of instructions: “write 3”, “remove 3” and “read 10”.

- **Sample 2 Input**

```
4 10 11
request 4
request 5
release* 4
release* 5
```

```
request 1
request 6
release* 1
release* 6
request 4
request 7
request 1
```

- **Sample 2 Output**

```
read 4
read 5
read 1
read 6
remove 5
read 7
```

- **Usage of LLM and Academic Integrity**

You must implement the core logic yourself. LLMs may be used only for programming-language syntax/grammar guidance.

If you achieve a high score on this lab, we will infer that you have a strong understanding of buffer replacement. If you then receive a very low score on the corresponding written-exam question, we will apply an additional penalty to your lab grade. In extreme discrepancies (e.g., full marks on the lab but zero on the written question), we will treat this as academic misconduct and report it to the AR.